Case 02 · Research and POC: Private Exposure Registry

Following the Magshimim "Research and POC" template: basic research, five key features, and the first POC (goals, plan, design, implementation and conclusions). Every POC was written in Python and actually run, and the numbers here are its output. Each POC states explicitly what is real and what is simulated.

Basic research: sequence of events

- **Local preparation:** each participant (a fund, or a regulator on behalf of its funds) computes its data locally to a uniform template: dollar exposure by rate bucket and by sector, plus its leverage bucket. The raw data never leave.
- **Splitting:** every number is split into three random shares that sum to the number (secret sharing). Each share goes to a different server.
- **Aggregation:** each server adds up the shares it received from all participants. No server sees any participant's value, only noise.
- **Threshold check:** participants are counted for each cell (the count is computed the same way). A cell with fewer than 5 participants is not revealed.
- **Privacy noise:** each server adds controlled noise (differential privacy) to its share, so the aggregate answer does not give away a single participant.
- **Opening:** only the aggregate result is opened, when the three servers combine their shares.

Basic research: system components

- **Participant Client:** computes the template locally and splits it.
- **Compute Servers (three, at different organisations):** aggregate shares and add noise.
- **Threshold Gate:** blocks small cells.
- **Release Service:** combines shares and publishes the result to the regulator.
- **Query Catalog:** a closed list of permitted questions, so that no one can ask a question designed to expose someone.

Basic research: security challenges

- **Collusion between servers:** if all servers combine their shares, they see everything. The defence is organisational: each server sits with a different body.
- **Leakage through the result:** in a cell with two participants, each can subtract itself from the sum and learn the other. Hence a minimum threshold and noise.
- **A lying participant:** the method protects secrecy, not truth. A participant can report wrong numbers, and no one will see. This is a serious open question.
- **Availability:** in a scheme where all three shares are needed, one server going down stops everything.

Basic research: comparison with the leading competitor (Form PF and ordinary regulatory reporting)

- **Secrecy:** with Form PF the regulator sees everything. Here no one sees a single data point.
- **Sharing between authorities:** with Form PF the data stay at the SEC. Here several authorities can compute together without moving data.
- **Depth:** Form PF allows any analysis on the raw data. Here only the catalog questions.
- **Reporting reliability:** with Form PF the regulator can audit and check. Here it is harder to check data no one sees.

Conclusion of the comparison: we give up flexibility and auditability, and gain sharing that is impossible today.

Basic research: ethical issue

A system that promises "no one sees" must keep that promise. If a leak is discovered, it will damage trust in every tool of this kind. Response: a closed query catalog, a minimum threshold, controlled noise, and public documentation of what does leak (for example the number of participants in each cell).

Five key features

- **Reporting template:** Input: the participant's loan book. Output: a uniform vector (rate buckets times sectors, plus a leverage bucket). How: local grouping and clipping of each cell to a cap, to bound any single participant's influence.
- **Secret sharing:** Input: the vector. Output: three random vectors. How: modular addition over a prime field (2 to the power 61, minus 1), with a cryptographic random generator.
- **Server aggregation:** Input: shares from all participants. Output: a partial sum. How: modular addition only, without seeing values.
- **Threshold gate and noise:** Input: participant count per cell, partial sum. Output: an allowed or blocked cell, and a noised share. How: threshold K=5, Laplace noise scaled to the cap divided by epsilon.
- **Opening and report:** Input: shares from the three servers. Output: answers to catalog questions (such as the share of exposure above 5%, sector concentration, a leverage histogram). How: combining the shares and computing the ratios.

The POC: research question

Can the exposure data of hundreds of participants be combined into an aggregate picture that is accurate enough, with no server seeing a single data point and without the result giving any participant away, in reasonable time?

The POC: goals

- **Correctness:** without noise, the result must equal the true sum exactly.

- **Secrecy:** a single server cannot tell a value of 0 from a value of one million. Two servers out of three cannot reconstruct a value.
- **Output protection:** a cell with fewer than 5 participants is blocked. Measure the precise cost of noise at three privacy levels.
- **Performance:** 500 participants in reasonable time.

The POC: technologies and limits

- Pure Python, with secrets (cryptographic randomness) and modular arithmetic. No libraries, so it runs in the browser.
- **Important limit:** the participants are synthetic (500 random funds across 7 sectors, one of them deliberately rare). The cryptography is real but simple: "honest but curious" servers (they follow the protocol and may peek), with no protection against a malicious server, and all three servers run in the same process.
- **Design decision:** participants bucket their data locally, so the servers only need to add. Comparisons ("is the rate above 5%?") are very expensive in joint computation, and local bucketing avoids them entirely.
- **Out of scope:** protection against a malicious server (MACs as in SPDZ), a real network, verifying that a participant reported the truth.

The POC: flow

Loan book (stays with the participant) → bucketed and clipped vector → split into three shares → server A / server B / server C (aggregation) → participant count → threshold gate → noise → combine → catalog answers.

The POC: results

- **Correctness:** without noise, every cell came out exactly equal to the true sum.
- **Performance:** 500 participants, 33 cells each: splitting, aggregation and opening in 0.11 seconds.
- **Secrecy against a single server:** the distribution of shares a server sees for a value of 0 and for a value of one million is identical within sampling noise (maximum gap of 0.65% between histogram bins, over 20,000 samples).
- **Secrecy against two servers:** combining two shares out of three returned a huge random number. Combining all three returned exactly the value (777).
- **Threshold gate:** three cells in the rare sector were blocked automatically. In an attack example, a participant in a cell of two would have learned the other's value (65) without the threshold; with the threshold the cell is blocked.
- **The price of privacy:** at epsilon 1 (strong privacy) the median cell error is 2.2% and the maximum 7.9%. At epsilon 4, 0.6% and 2%. At epsilon 16, 0.1% and 0.5%. The headline question, what share of exposure carries a rate above 5%, came out at 17.5% against a true 16.8% at epsilon 1, and 17% at epsilon 4. Even with strong privacy, the answer is useful.

- The leverage histogram at epsilon 4 came out almost identical to the truth (an error of at most one participant per cell).

The POC: what is real and what is simulated

Real: the secret sharing, aggregation, threshold and noise, and the proof that they work together. Simulated: the participants and their data, and the servers running on one machine. Performance will be slower over a real network, and much slower with protection against malicious servers.

The POC: conclusions

✅ Concept proven: the exposure data of hundreds of participants can be combined into an accurate aggregate picture, with no server seeing a single data point, in under a second. But the POC exposed two questions the cryptography does not answer. First: the method protects secrecy, not truth. A participant can report wrong numbers. Second: who runs the three servers, and what happens when one of them goes down. Both move to the spec.